

2.28

Simple Selection Control Structure

Subtitle



Control Structure

- **Control structures** form the basic entities of a “**structured programming language**”.
- C, C++ or Java are all structured programming languages.
- *Control structures are used to alter the flow of execution of the program.*



Control Structure

- Why do we need to alter the program flow ? The reason is “**decision making**”!
- In life, we may be given with a set of option like doing “Electronics” or “Computer science”.
- We do make a decision by analyzing certain conditions (like our personal interest, scope of job opportunities etc).
- With the decision we make, we alter the flow of our life’s direction.
- This is exactly what happens in a C program.
- We use control structures to make decisions and alter the direction of program flow in one or the other path(s) available.

Type of Control Structure

- ✓ There are **3 TYPES** of **control structures** available in C

- ① • Sequence structure (**straight line paths**)
- ② • Selection structure (one or many branches)
- ③ • Loop structure (repetition of a set of activities)

① Sequence
② selection
③ loop

Control statements in C to implement control structures

- All program processes can be implemented with these 3 control structures only.
- To implement these “control structures” in a C program, the language provides ‘control statements’.
- So to implement a particular control structure in a programming language, we need to learn how to use the relevant control statements in that particular language.

The Control Statements

- The control statements are:-

- **if**

- **if...else**

- **switch**

- **while**

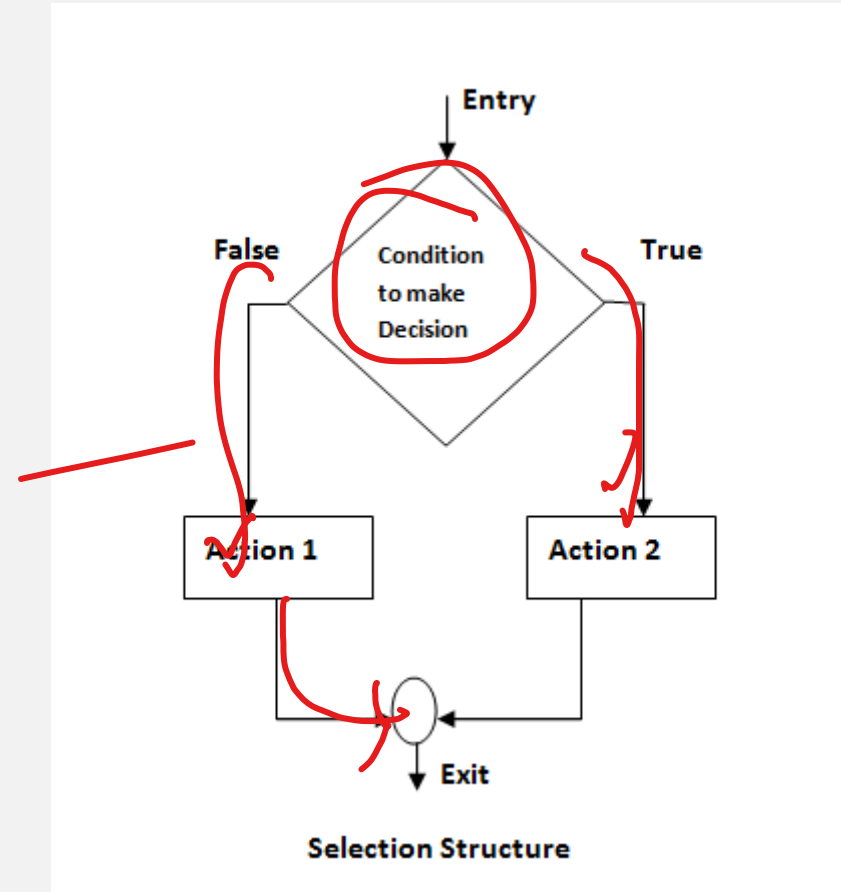
- **do...while**

- **for**

Control Structure Commands

- **Selection** structures are implemented using `if` , `if else` and `switch` statements.
- **Looping** structures are implemented using `while`, `do while` and `for` statements

Selection Structure Flow Chart (simple if command)



Selection Structure

- Selection structures are used to perform **'decision making'**.
- The branch the program flow based on the outcome of decision making.
- Selection structures are implemented in C with `if`, `if..else` and `switch` statements.
 - `if` and `if..else` statements are 2 way branching statements
 - `switch` is a multi branching statement.

The Syntax if statement

- *if (expression)*
 {
 statement 1;
 statement 2;
 }
 statement 3;// Program control is transferred directly to this line, if
 the expression is FALSE
 statement 4;
- *// This expression is evaluated. If expression is TRUE statements inside
 the braces will be executed*

The Syntax if statement

- The expression given inside the brackets after **if** is evaluated first.
- If the expression is **TRUE**, then statements inside the curly braces that follow `if(expression)` will be executed.
- If the expression is **FALSE**, the statements inside curly braces will not be executed and program control goes directly to statements after curly braces.
- But...statement 3 and 4 will be executed after the curly braces completed.

Example program to demo 'if' statement

- **Problem:-**

- A simple example program to demo the use of `if`, `if...else` and `switch` is shown here.
 - An integer value is collected from user.
 - If the integer entered by user is 1 – output on screen “Bahasa Melayu”.
 - If the integer is 2 – output “English”,
 - If the integer is 3 output “France”.
 - If the user enters some other value – output “Invalid Input”.

Example program to demo 'if' statement

```
#include
void main()
{
    int num;
    printf("Hello user, Select a language");
    scanf("%d",&num); // Collects the number from user
    if(num==1)
        printf("Bahasa Melayu");
        printf("Terma Kasih");

    if(num==2){
        printf("English");
        printf("TQ");
    }
    if(num==3){
        printf("France");
    }
    if(num > 3){
```



Example program to demo 'if' statement

- Question?
 - Write a selection statement to check the user enters a value 1 and 2 only, otherwise display "WRONG ENTRY".

- Answer.

- ```
if (num <= 1) && (num > 3)
{
 printf("WRONG ENTRY");
}
```

# The Syntax if...else statement

```
if(expression 1)// Expression 1 is evaluated. If TRUE, statements inside the curly braces are executed.
```

```
{ //If FALSE program control is transferred to immediate else if statement.
 statement 1;
 statement 2;
}
```

```
else if(expression 2)// If expression 1 is FALSE, expression 2 is evaluated.
{
 statement 1;
 statement 2;
}
```

```
else if(expression 3) // If expression 2 is FALSE, expression 3 is evaluated
{
 statement 1;
 statement 2;
}
```

```
else // If all expressions (1, 2 and 3) are FALSE, the statements that follow this else (inside curly braces) is executed.
```

```
{
 statement 1;
 statement 2;
}
```

```
other statements;
```



# The Syntax `if...else` statement

- The execution begins by evaluation expression 1.
- If it is **TRUE**, then statements inside the immediate curly braces is evaluated.
- If it is **FALSE**, program control is transferred directly to immediate *else if* statement.
- Here expression 2 is evaluated for TRUE or FALSE.
- The process continues.
- If all expressions inside the different *if* and *else...if* statements are **FALSE**, then the last **else** statement (without any expression) is executed along with the statements 1 and 2 inside the curly braces of last **else** statement.

# Example program to demo 'if...else' statement

```
• #include
void main()
{
 int num;
 printf("Hello user, Enter a number");
 scanf("%d",&num); // Collects the number from user
 if(num==1) {
 printf("UNITED STATES");
 }
 else if(num==2) {
 printf("SPAIN");
 }
 else if(num==3) {
 printf("INDIA");
 }
 else {
 printf("WRONG ENTRY");
 }
}
```

# switch..case statement

- Instead of writing many if..else statements, you can use the `switch` statement.
- The `switch` statement selects one of many code blocks to be executed:

## Syntax

```
switch (expression) {
 case x:
 // code block
 break;
 case y:
 // code block
 break;
 default:
 // code block
}
```

# Example 1

- Convert the previous if..else code to switch..case statement

```
#include <stdio.h>

void main() {
 int num;
 printf("Hello user, Enter a number: ");
 scanf("%d", &num);
 switch(num) {
 case 1:
 printf("UNITED STATES");
 break;
 case 2:
 printf("SPAIN");
 break;
 case 3:
 printf("INDIA");
 break;
 default:
 printf("WRONG ENTRY");
 }
}
```

## Example 2

- uses the weekday number to calculate the weekday name

```
int day = 5;

switch (day) {
 case 1:
 printf("Monday");
 break;
 case 2:
 printf("Tuesday");
 break;
 :
 :
 case 5:
 printf("Friday");
 break;
 case 6:
 printf("Saturday");
 break;
 case 7:
 printf("Sunday");
 break;
} // Outputs "Friday" (day 5)
```

# The **break** Keyword

- When C reaches a **break keyword**, it breaks out of the switch block.
- This will stop the execution of more code and case testing inside the block.
- **When a match is found**, and the job is done, it's time for a break. There is no need for more testing.
- A break can save a lot of execution time because it "ignores" the execution of all the rest of the code in the switch block.

# The **default** Keyword

- The default keyword specifies some code to run if there is no case match.
- Note: The default keyword must be **used as the last statement** in the switch, and it does **not need a break**.

```
int day = 4;

switch (day) {
 case 6:
 printf("Today is Saturday");
 break;
 case 7:
 printf("Today is Sunday");
 break;
 default:
 printf("Looking forward to the Weekend");
}

// Outputs "Looking forward to the Weekend"
```

# Selection Structure



## Recall

---

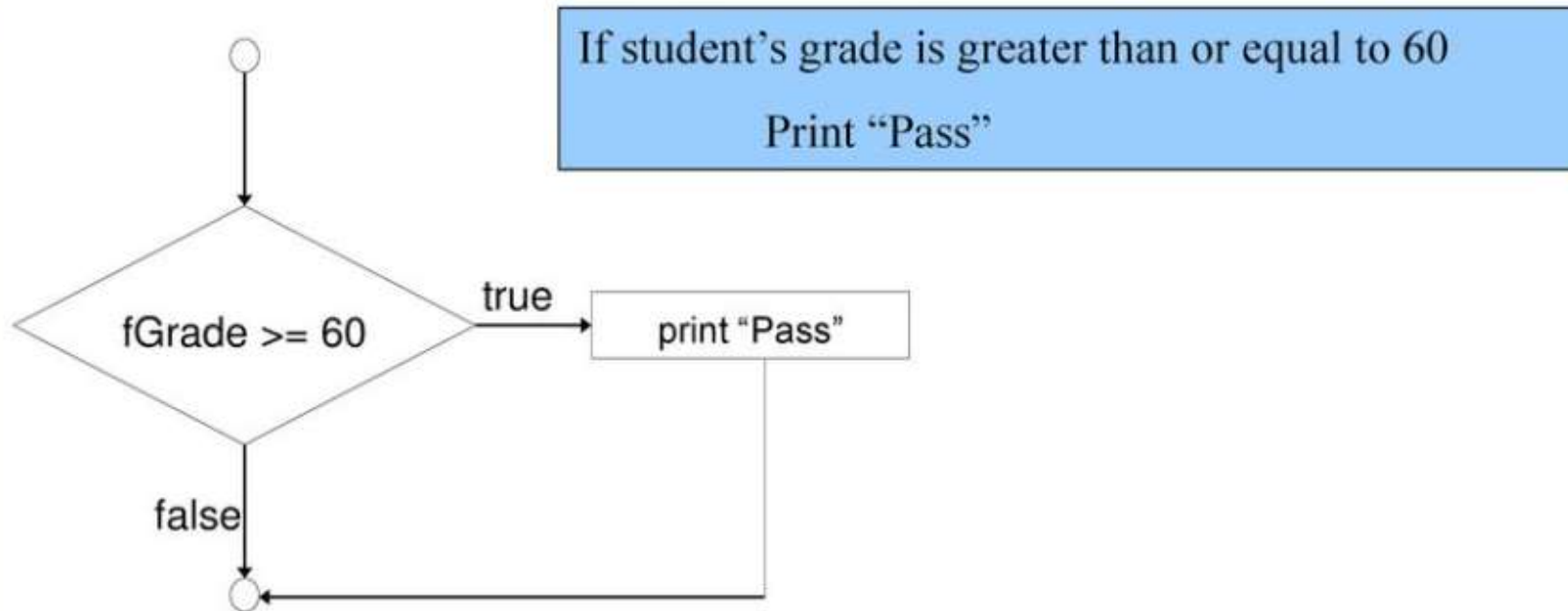
### Recall.. Selection Structure

- Used to choose among alternative courses of action
  - C has three types: **if**, **if..else**, and **switch**
- 

## The *if* selection structure

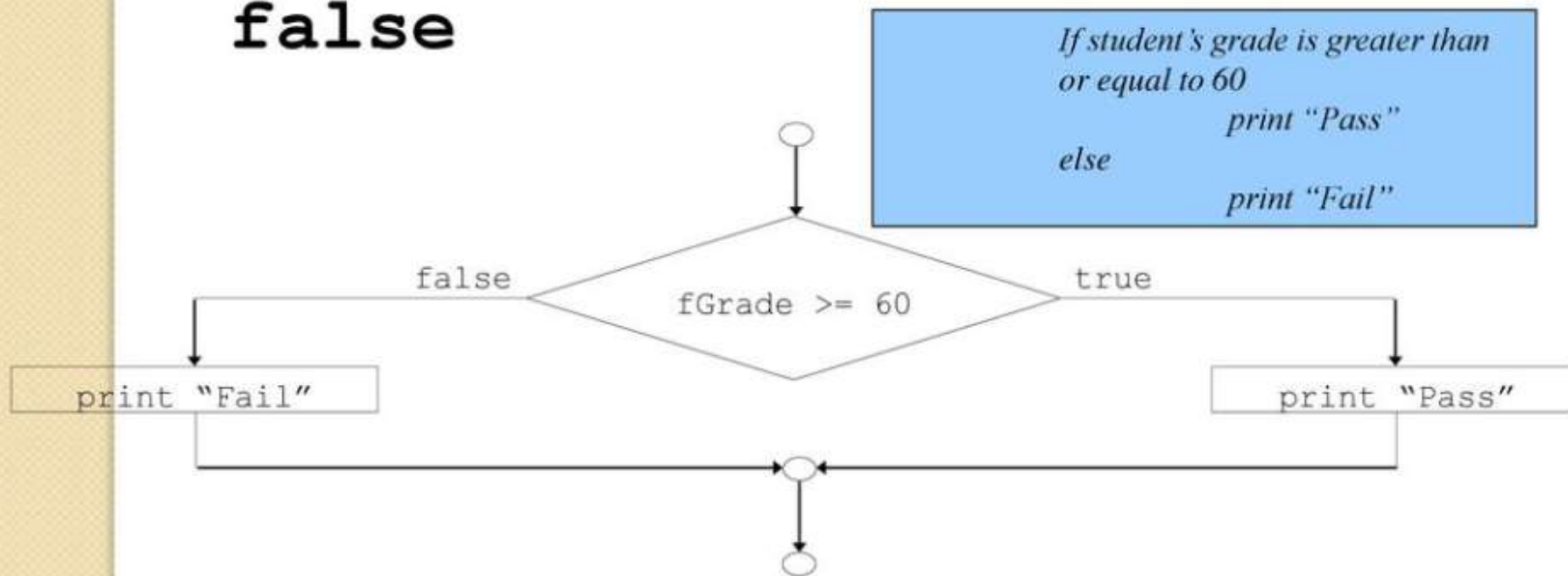
---

- **if** structure is a single-entry/single-exit structure



## The *if...else* selection structure

- Specifies an action to be performed both when the condition is **true** and when it is **false**



## Selection Statement

---

- Used to control the flow of a program
- Also called as **decision** or **branches**
- Branches are conditions or choices used to enable selection of program flow

## Type of selection

---

- One-way selection = `if`
- Two-way selection = `if..else`
- Multi-selection
- Nested if
- Switch structure = `switch`

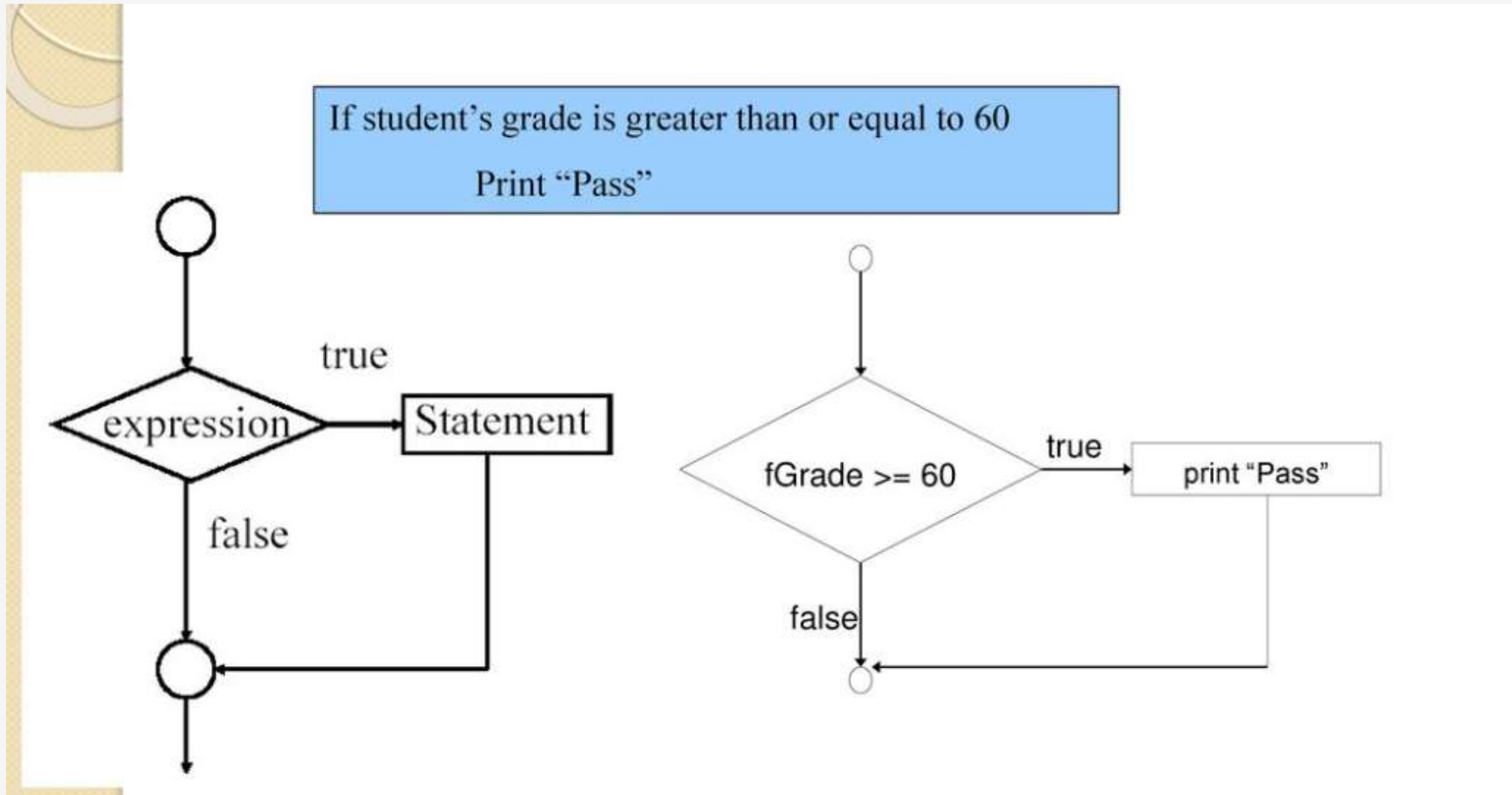
## One – Way selection = if

---

- In C, a **condition** is represented by a **logical (Boolean) expression**
- **true** and **false** are logical (Boolean) values
- The syntax of one-way selection is:
  - **if (expression) statement;**
- If the **value** of the expression is **true**, statement is executed;
- if **false**, statement is not executed and the computer goes on to the next statement in the program.

## One – Way selection = if

---



One – Way selection = if

---

....

```
if(fGrade >= 60)
 printf("Pass");
```

....

....



## One – Way selection = if

---

- Another example:

```
char cGrade='F';
```

```
.....
```

```
if(fMarkah >= 90)
```

```
 cGrade = 'A';
```

```
.....
```

```
.....
```

```
printf("Grade is : %c\n", cGrade);
```

## One – Way selection = if

---

- Another example:
  - if (temperature is greater than 70 degree and it is not raining)
    - recommended activity is golfing

```
bool rain=false;
```

```
...
```

```
if((fTemp > 70) && !(rain))
```

```
 printf("recommended activity is golfing");
```

One – Way selection = if

---

- **Common Errors**

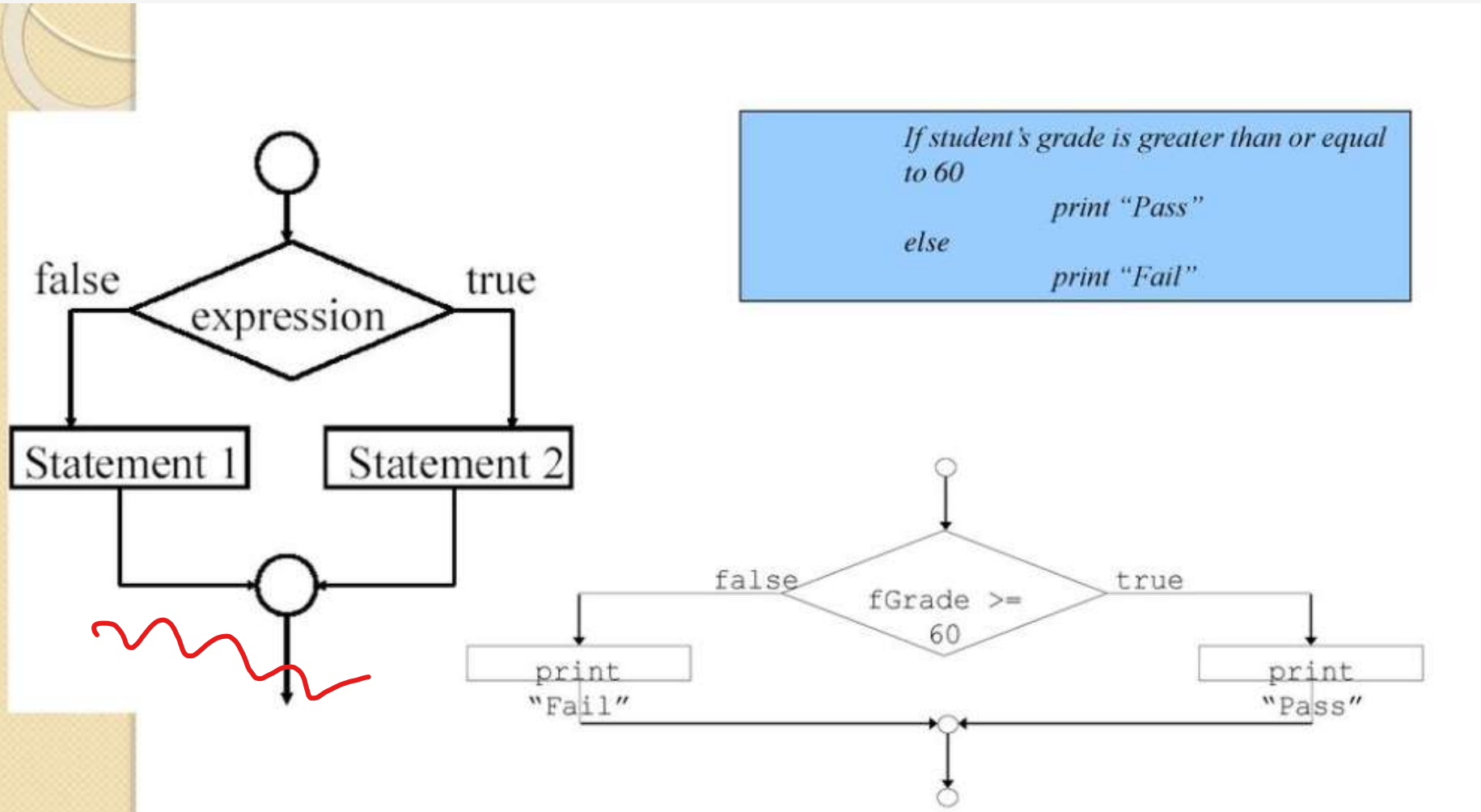
- `if fScore >= 90`      *//no parentheses*  
    `cGrade = 'A';`
- `if(fScore >= 90);` *//; not here*  
    `cGrade = 'A';`

## Two – Ways selection = if ... else

---

- The syntax of two-way selection is:
  - if (expression)  
    statement1;  
else  
    statement2;
- If the **value** of the expression is **true**, statement1 is executed;
- if **false**, statement2 is executed

## Two – Ways selection = if ... else



## Two – Ways selection = if ... else

---

.....

```
if(fGrade >=60)
```

```
 printf("Pass");
```

```
else
```

```
 printf("Fail");
```

.....

## Two – Ways selection = if ... else

---

- Another example:

```
if (fHour > 40.0) //Line 1
 fWages = 40.0 * fRate + 1.5 * fRate * (hour -
 40.0); //Line 2
else //Line 3
 fWages = fHour * fRate;
 //Line 4
```

- If fHour is 50, then the statement at **Line 2** is executed
- If fHour is 30, then the statement at **Line 4** is executed

## Multi-selection = if ... else if ... else

---

- The syntax is:

```
if(exp1)
 stmt1;
else if(exp2)
 stmt2;
else if(exp3)
 stmt3;
...
else
 stmt n;
```

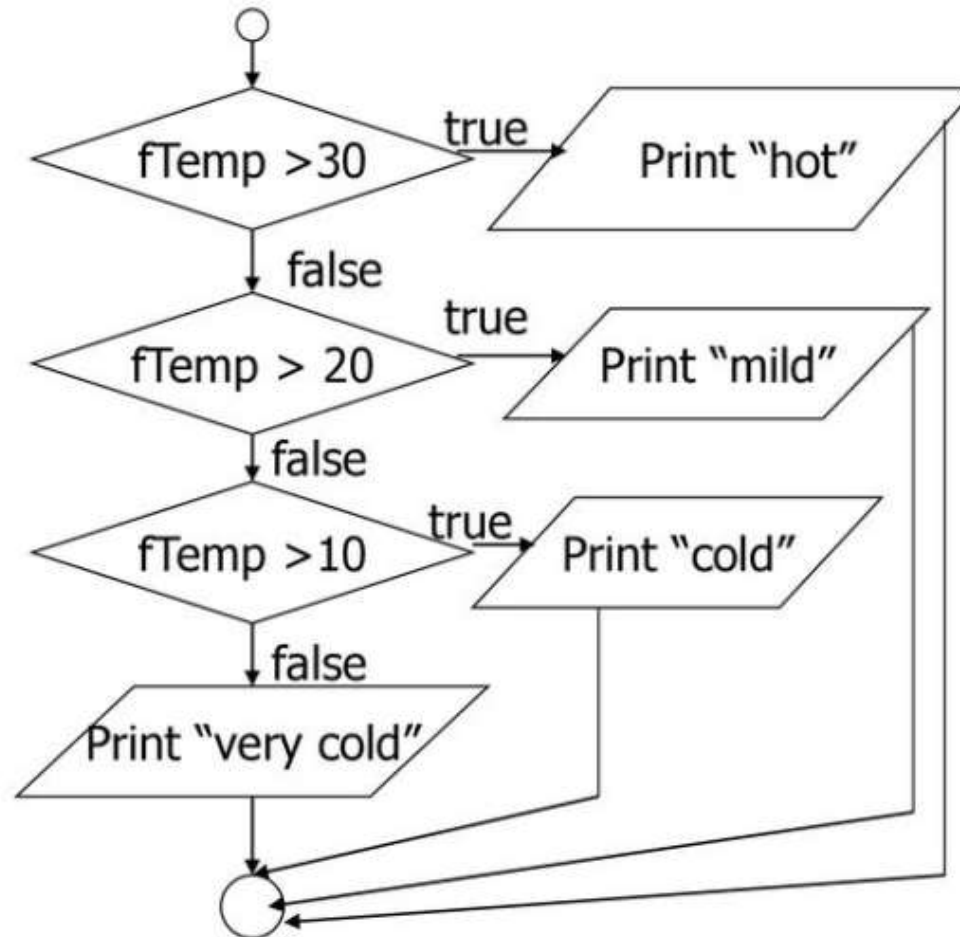
An if-else if control structure shifts program control, step by step, through a series of statement blocks.



## Multi-selection = if ... else if ... else

• E.g.

| temp     | display   |
|----------|-----------|
| >30 °c   | hot       |
| 20-30 °c | mild      |
| 10-20 °c | cold      |
| <10 °c   | very cold |



Multi-selection = if ... else if ... else

---

```
if(fTemp > 30)
 printf("hot\n");
else if(fTemp >20)
 printf("mild\n");
else if(fTemp >10)
 printf("cold\n");
else
 printf("very cold\n");
```

## Compound (block of) Statement

---

- A **compound statement** (also called a **block of statements**) takes the form of
  - {
  - statement 1; statement 2;
  - .
  - .
  - .
  - statement n;
  - }
- It is considered a single statement

## Compound (block of) Statement

---

- Example:  
if (iAge > 18)  
{  
    printf("Eligible to vote\n");  
    printf("No longer a minor\n");  
}  
else  
{  
    printf("Not eligible to vote\n");  
    printf("Still a minor\n");  
}

# Nested if

---

- When one control statement is within another, it is said to be **nested**
- ```
if(exp1)
    if(exp2)
        statement1; OR
```
- ```
if(exp1)
{
 statement1;
 if(exp2)
 statement2;
}
```

## Nested if

---

- Example:

```
if (fTemperature >= 50)
{
 if (fTemperature >= 80)
 printf("Good day for swimming.\n");
 else
 printf("Good day for golfing.\n");
}
else
 printf("Good day to play tennis.\n");
```



# Nested if

- Another example

```
#include <stdio.h>
void main (void)
{
 int iDay;
 float fTime;

 printf ("Type the day and time of interest\n\n");
 scanf ("%d %f ", &iDay, &fTime);

 if (iDay <= 5)
 {
 if (fTime <= 9.00)
 printf (" Work \n\n");
 else
 printf (" Relax \n\n");
 }
 else
 {
 if (fTime <= 8.00)
 printf (" Sleep \n\n");
 else
 printf (" Have Fun \n\n");
 }
}
```

## Output

Type the day and time of interest

Keyboard input      3 10.00  
                         Relax

## Conditional Operator (?:)

---

- 
- The syntax of using the conditional operator is:  
`expression1 ? expression2 : expression3;`
  - This is called a **conditional expression**.
  - The statement:  

```
if (a >= b)
 max = a;
else
 max = b;
```
  - Is equivalent to the statement:  
`max = (a >= b) ? a : b;`



# switch Structure

---

- Similar to if-else if control structure
- The general form (syntax):

```
switch (expression)
{
 case value1: statements1; break;
 case value2: statements2; break;
 .
 .
 .
 case valuen: statementsn; break;
 default: statements;
}
```

## switch Structure

---

- The **break** statement has a special meaning and may or may not appear after each statement.
- In C, **switch**, **case**, **break**, and **default** are reserved words.
- In a **switch** structure, first the expression is evaluated. The value of the expression is then used to perform the corresponding action.

## switch Structure

---

- The expression is usually an identifier.
- The value of the expression can be only integral.
- The expression is sometimes called the **selector**. Its value determines which statement is selected for execution.
- A particular **case** value should appear only once.
- One or more statements may follow a case label, so you do not need to use braces to turn multiple statements into a single compound statement.
- The **break** statement may or may not appear after each statement.

# switch Structure

---

- **Example:**

```
switch (cGrade)
{
 case 'A': printf("The grade is A."); break;
 case 'B': printf("The grade is B."); break;
 case 'C': printf("The grade is C."); break;
 case 'D': printf("The grade is D."); break;
 case 'F': printf("The grade is F."); break;
 default: printf("The grade is invalid.");
}
```

- where, cGrade is a variable of the type **char**.  
If the value of cGrade is, say 'A', the output is  
**The grade is A.**

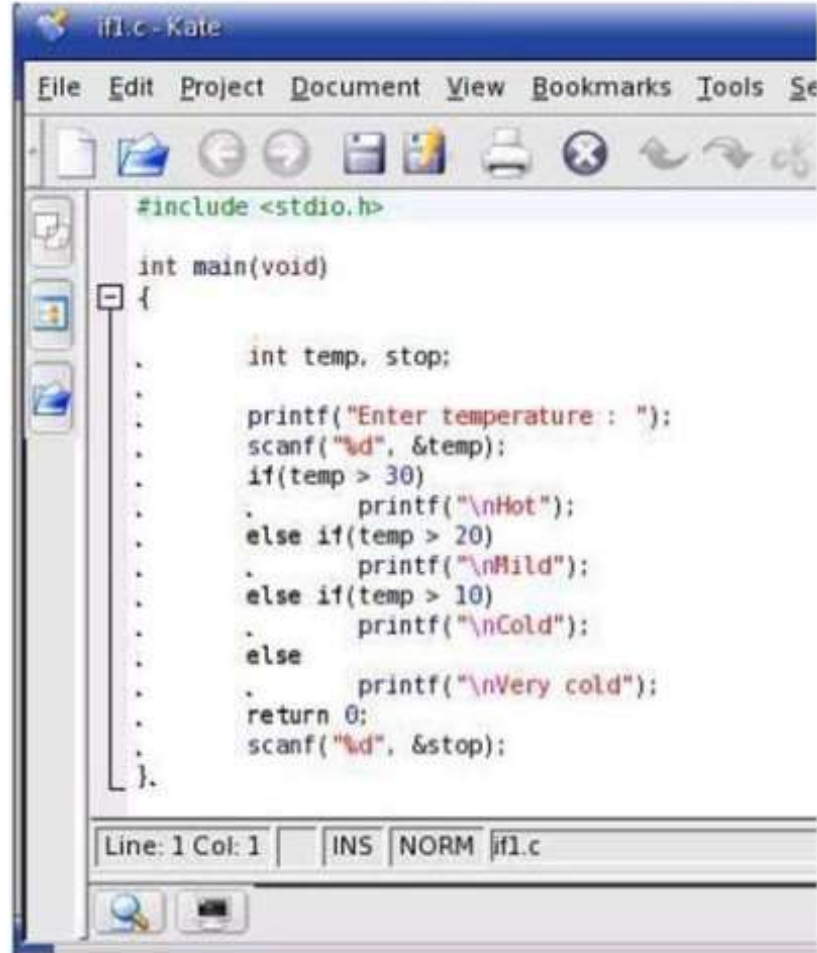


# switch Structure

---

- The switch statement executes according to the following rules:
  - When the value of the expression is matched against a case value (also called a label), the statements execute until either a break statement is found or the end of the switch structure is reached.
  - If the value of the expression does not match any of the case values, the statements following the default label execute. If the switch structure has no default label, and if the value of the expression does not match any of the case values, the entire switch statement is skipped.
  - A break statement causes an immediate *exit* from the switch structure

# What's wrong?

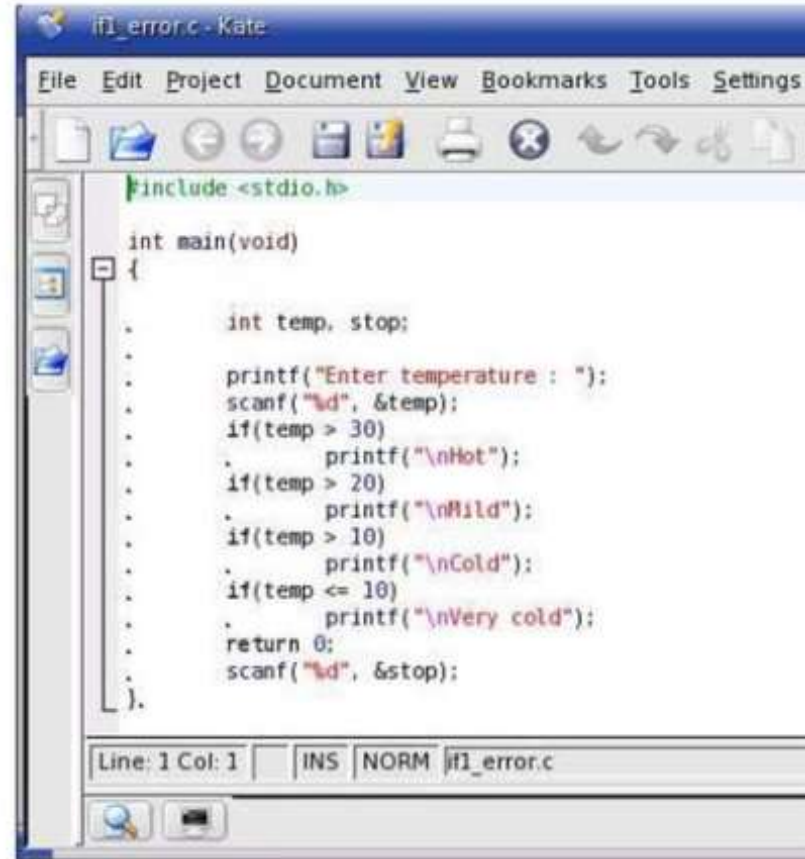


```
#include <stdio.h>

int main(void)
{
 int temp, stop;

 printf("Enter temperature : ");
 scanf("%d", &temp);
 if(temp > 30)
 printf("\nHot");
 else if(temp > 20)
 printf("\nMild");
 else if(temp > 10)
 printf("\nCold");
 else
 printf("\nVery cold");
 return 0;
 scanf("%d", &stop);
}
```

Line: 1 Col: 1 | INS | NORM | if1.c



```
#include <stdio.h>

int main(void)
{
 int temp, stop;

 printf("Enter temperature : ");
 scanf("%d", &temp);
 if(temp > 30)
 printf("\nHot");
 if(temp > 20)
 printf("\nMild");
 if(temp > 10)
 printf("\nCold");
 if(temp <= 10)
 printf("\nVery cold");
 return 0;
 scanf("%d", &stop);
}
```

Line: 1 Col: 1 | INS | NORM | if1\_error.c



Thank you